end of the range even when there's a match toward the beginning, which might be surprising if you're not expecting it.

OK, so you've managed to parallelize `std::find`. As I stated at the beginning of this section, there are other similar algorithms that can complete without processing every data element, and the same techniques can be used for those. We'll also look further at the issue of interrupting threads in chapter 9.

To complete our trio of examples, we'll go in a different direction and look at `std::partial_sum`. This algorithm doesn't get a lot of press, but it's an interesting algorithm to parallelize and highlights some additional design choices.

8.5.3. A parallel implementation of std::partial_sum

`std::partial_sum` calculates the running totals in a range, so each element is replaced by the sum of that element and all the elements prior to it in the original sequence. Thus the sequence 1, 2, 3, 4, 5 becomes 1, (1+2)=3, (1+2+3)=6, (1+2+3+4)=10, (1+2+3+4+5)=15. This is interesting to parallelize because you can't just divide the range into chunks and calculate each chunk independently. For example, the initial value of the first element needs to be added to every other element.

One approach to determining the partial sum of a range is to calculate the partial sum of individual chunks and then add the resulting value of the last element in the first chunk onto the elements in the next chunk, and so forth. If you have the elements 1, 2, 3, 4, 5, 6, 7, 8, 9 and you're splitting into three chunks, you get {1, 3, 6}, {4, 9, 15}, {7, 15, 24} in the first instance. If you then add 6 (the sum for the last element in the first chunk) onto the elements in the second chunk, you get {1, 3, 6}, {10, 15, 21}, {7, 15, 24}. Then you add the last element of the second chunk (21) onto the elements in the third and final chunk to get the final result: {1, 3, 6}, {10, 15, 21}, {28, 36, 55}.